Chapter 4. 외부 API 연동하기

학습 목표

이번 장에서는 외부 API를 호출하는 Tool을 만들고, AI Agent와 연동하여 실시간 정보를 조회하는 방법을 학습합니다.

학습이 끝나면 다음 내용을 수행할 수 있습니다.

- REST API 개념 이해
- Python에서 API 호출
- API 호출 결과 처리
- Agent와 API Tool 연동
- API Key 등 민감 정보 관리(.env)

이 장의 용어

용어	영문 · 약어 풀이	설명
API	Application Programming Interface	프로그램끼리 기능과 데이터를 주고받기 위해 정해 둔 약속(접점)
REST	Representational State Transfer	HTTP를 이용해 데이터를 주고받는 API 설계 방식. 가장 널리 쓰임
HTTP	HyperText Transfer Protocol	웹 브라우저와 서버가 데이터를 주고받는 통신 규약. HTTPS는 암호화된 HTTP
요청 / 응답	Request / Response	클라이언트가 서버에 보내는 것이 요청, 서버가 돌려주는 것이 응답
HTTP Method	메서드, 요청 방식	요청의 목적을 나타내는 동사. GET(조회), POST(생성), PUT(수정), DELETE(삭제)
URL	Uniform Resource Locator, 인터넷 주소	요청을 보낼 대상의 주소. `https://서버/경로?파라미터` 구조
엔드포인트	Endpoint	API에서 특정 기능을 담당하는 URL 경로. `/v1/forecast`가 날씨 조회 엔드포인트
쿼리 파라미터	Query Parameter	URL의 `?` 뒤에 `이름=값&이름=값` 형태로 붙여 요청 조건을 전달하는 값

용어	영문 · 약어 풀이	설명
JSON	JavaScript Object Notation	`{"이름": 값}` 형태의 데이터 표기 형식. API 응답의 표준. Python에서는 딕셔너리로 다룸
상태 코드	Status Code	응답의 결과를 나타내는 숫자. 200 성공, 404 없음, 500 서버 오류
requests	—	Python에서 HTTP 요청을 보내는 대표 라이브러리. `requests.get(url)` 한 줄로 호출
객체	Object	관련 데이터와 기능을 하나로 묶은 단위. Response 객체는 응답 데이터(`status_code`, `text`)와 기능(`json()`)을 함께 가짐
헤더	Header	요청·응답 본문과 별도로 보내는 부가 정보. 인증 정보, 데이터 형식 등을 담음
위도 / 경도	Latitude / Longitude	지구 위 위치를 나타내는 좌표. 위도는 남북(적도 0°), 경도는 동서. 서울은 약 37.57, 126.98
예외 / 예외 처리	Exception / Exception Handling	예외는 실행 중 발생하는 오류, 예외 처리는 `try` / `except`로 오류를 잡아 프로그램이 멈추지 않게 하는 것
timeout	타임아웃, 제한 시간	응답을 기다리는 최대 시간(초). 넘으면 예외 발생. 서버가 멈췄을 때 무한 대기를 막음
Bearer 토큰	Bearer Token, 소지자 토큰	"이 토큰을 가진(bearer) 사람의 요청"임을 나타내는 인증 방식. 헤더에 `Authorization: Bearer 키` 형태로 담음
JSON 변환 가능	JSON-serializable, 직렬화 가능	문자열·숫자·불·리스트·딕셔너리처럼 JSON 텍스트로 바꿀 수 있는 값. Tool의 반환값은 이 형태여야 함

1. REST API

1.1. API란?

지금까지 우리는 AI Agent가 여러 Tool을 사용할 수 있다는 것을 학습했습니다.

예를 들어

- 현재 시간 조회
- 계산
- 주사위 굴리기

등의 Tool을 만들어 보았습니다.

하지만 이러한 Tool은 모두 **프로그램 내부에서 수행할 수 있는 작업**이었습니다.

실제 업무에서는 대부분의 데이터가 외부 시스템에 저장되어 있습니다.

예를 들어

- 날씨 정보
- 환율 정보
- 주가 정보
- 지도 정보
- 고객 정보
- 주문 정보

등은 대부분 다른 시스템이 관리하고 있습니다.

AI Agent가 이러한 정보를 사용하려면 외부 시스템과 통신해야 합니다.

이때 사용하는 것이 **API(Application Programming Interface)** 입니다.

API는

> **프로그램과 프로그램이 서로 데이터를 주고받기 위한 약속(인터페이스)** 입니다.

1.2. 사람으로 비유하면

API를 이해하는 가장 쉬운 방법은 식당을 생각해 보는 것입니다.

손님은 음식을 직접 만들지 않습니다. 주방에도 들어가지 않습니다. 대신 직원에게 주문합니다.

```
손님

↓

직원

↓

주방

↓

직원

↓
```

```
손님
```

여기서 직원은 손님과 주방을 연결하는 역할을 합니다.

API도 같은 역할을 합니다.

```
AI Agent

↓

API

↓

외부 시스템
```

AI는 직접 데이터베이스에 접근하는 것이 아니라 API를 통해 요청을 보냅니다.

1.3. API가 필요한 이유

예를 들어, 사용자가

```
서울 날씨 알려줘.
```

라고 질문했습니다.

현재 날씨는 기상청이나 날씨 서비스가 관리하고 있습니다.

AI는 그 데이터를 직접 가지고 있지 않습니다.

따라서

```
AI Agent

↓

Weather API

↓

현재 날씨 조회

↓
```

```
응답
```

과정을 수행해야 합니다.

1.4. REST API란?

API에도 여러 종류가 있습니다.

그중 가장 많이 사용하는 방식이 **REST API**입니다.

REST는 **Representational State Transfer**의 약자입니다.

용어는 어렵지만 실제로는

> **HTTP(HyperText Transfer Protocol, 웹 브라우저와 서버가 데이터를 주고받는 통신 규약)를 이용하여 데이터를 주고받는 방식**

이라고 이해하면 충분합니다.

웹 브라우저가 웹 서버와 통신하는 것처럼 프로그램도 HTTP를 이용하여 데이터를 주고받습니다.

1.5. REST API는 어떻게 동작할까?

예를 들어, 사용자가

```
서울 날씨 알려줘.
```

라고 질문했습니다.

AI는 다음과 같은 요청을 보냅니다.

```
GET

/weather?city=Seoul

↓

서버

↓

{
    "city": "Seoul",
    "temperature": 28,
```

```
    "condition": "Sunny"
}

↓

AI

↓

응답 생성
```

즉, REST API는 **요청(Request)** 과 **응답(Response)** 으로 이루어집니다.

1.6. Request와 Response

REST API는 항상 두 가지로 구성됩니다.

Request

클라이언트가 서버에 보내는 요청입니다.

```
서울 날씨 알려줘.

↓

GET /weather?city=Seoul
```

Response

서버가 반환하는 결과입니다.

```
{
  "city":"Seoul",
  "temperature":28
}
```

AI는 이 데이터를 읽어서 사용자에게 자연어로 설명합니다.

1.7. REST API에서 사용하는 HTTP Method

REST API는 요청의 목적에 따라 여러 가지 HTTP Method를 사용합니다.

이번 과정에서는 네 가지를 알아봅니다.

Method	의미	예시
GET	데이터 조회	현재 날씨 조회
POST	데이터 생성	회원 등록
PUT	데이터 전체 수정	회원 정보 수정
DELETE	데이터 삭제	회원 삭제

AI Agent에서는 가장 많이 사용하는 방식이 **GET**입니다.

예를 들어

- 날씨 조회
- 환율 조회
- 뉴스 조회

등은 모두 GET 요청을 사용합니다.

1.8. REST API 예시

다음과 같은 URL(Uniform Resource Locator, 인터넷 주소)이 있다고 가정해 보겠습니다.

```
https://api.example.com/weather
```

AI는 다음과 같이 요청합니다.

```
GET

https://api.example.com/weather?city=Seoul
```

여기서 `?` 뒤의 `city=Seoul`을 **쿼리 파라미터(Query Parameter)** 라고 합니다. "어떤 데이터를 원하는지"를 서버에 알려주는 값이며, 여러 개일 때는 `&`로 연결합니다.

서버는

```
{
  "city":"Seoul",
  "temperature":28,
  "condition":"Sunny"
}
```

를 반환합니다.

1.9. JSON이란?

대부분의 REST API는 데이터를 **JSON(JavaScript Object Notation)** 형식으로 전달합니다.

예를 들어

```
{
    "name":"홍길동",
    "age":20,
    "major":"AI"
}
```

JSON은 사람도 읽기 쉽고 프로그램도 처리하기 쉬운 데이터 형식입니다.

Python에서는 JSON을 딕셔너리(Dictionary) 형태로 다룰 수 있습니다.

1.10. AI Agent와 REST API

AI Agent에서는 REST API를 직접 호출하지 않습니다.

보통 Tool을 이용하여 호출합니다.

구조는 다음과 같습니다.

```
사용자

↓

Agent

↓

Weather Tool

↓

REST API

↓

외부 서버

↓
```

```
JSON

↓

Gemini

↓

응답
```

즉, REST API는 Tool 내부에서 사용됩니다.

실무 활용 사례

AI Agent에서 REST API는 매우 다양한 업무에 활용됩니다.

요청	API
현재 날씨	Weather API
환율 조회	Exchange Rate API
주가 조회	Stock API
배송 조회	택배 API
지도 검색	Google Maps API
번역	Translation API
GitHub 정보 조회	GitHub REST API

실무에서는 AI Agent가 이러한 API를 Tool을 통해 호출하여 사용자에게 필요한 정보를 제공합니다.

---

2. Python에서 API 호출

2.1. Python에서 REST API를 호출하는 방법

앞 절에서는 HTTP 통신이 **Request(요청)** 와 **Response(응답)** 로 이루어진다는 것을 학습했습니다.

이제 Python 프로그램에서 직접 HTTP Request를 보내고 Response를 받아보겠습니다.

Python에서는 `requests` 라이브러리를 이용하여 HTTP 요청을 매우 쉽게 보낼 수 있습니다.

동작 과정은 다음과 같습니다.

```
Python 프로그램

↓

requests

↓

HTTP Request

↓

REST API Server

↓

HTTP Response

↓

Response 객체

↓

Python 프로그램
```

`requests`는 Python과 외부 서버를 연결해 주는 대표적인 HTTP 라이브러리입니다.

2.2. requests 라이브러리 설치

먼저 `requests` 라이브러리를 설치합니다.

```
pip install requests
```

또는 프로젝트의 `requirements.txt` 파일에 추가합니다.

```
google-adk
python-dotenv
requests
```

이후 다음 명령어를 실행하면 필요한 라이브러리를 한 번에 설치할 수 있습니다.

```
pip install -r requirements.txt
```

설치가 완료되면 Python에서 `requests`를 사용할 수 있습니다.

2.3. 첫 번째 API 호출

이번 실습에서는 테스트용 공개 API를 사용합니다.

다음 API는 사용자 정보를 JSON 형식으로 반환합니다.

```
https://jsonplaceholder.typicode.com/users/1
```

프로젝트 루트에 `practice` 폴더를 만들고 그 안에 파일을 작성합니다.

`practice/api_call.py`

```python
import requests

url = "https://jsonplaceholder.typicode.com/users/1"

response = requests.get(url)

print(response)
```

실행

```
python practice\api_call.py
```

> **참고**
> 프로젝트 루트(`my-adk`)에서 가상환경이 활성화된 상태(`(venv)` 표시)로 실행합니다.
> Windows 경로는 역슬래시(`\`)로 구분합니다.

실행 결과는 다음과 같습니다.

```
<Response [200]>
```

`200`은 요청이 정상적으로 처리되었다는 의미입니다.

하지만 아직 데이터는 출력되지 않았습니다. 현재는 **Response 객체**만 출력한 상태입니다.

2.4. Response 객체 이해하기

API를 호출하면 `requests.get()`은 **Response 객체(Object, 관련 데이터와 기능을 하나로 묶은 단위)**를 반환합니다.

```
response = requests.get(url)
```

Response 객체에는 서버가 반환한 다양한 정보가 저장됩니다.

대표적인 속성은 다음과 같습니다.

속성	설명
`status_code`	HTTP 상태 코드
`text`	응답 내용을 문자열로 반환
`json()`	JSON 데이터를 Python 객체로 변환
`headers`	응답 헤더 정보

즉, Response 객체는 **서버의 응답을 담고 있는 상자**라고 생각하면 됩니다.

2.5. 상태 코드 확인하기

요청이 성공했는지 확인하려면 `status_code`를 사용합니다.

`practice/api_call.py`

```
import requests

url = "https://jsonplaceholder.typicode.com/users/1"

response = requests.get(url)

print(response.status_code)
```

실행 결과

```
200
```

대표적인 상태 코드는 다음과 같습니다.

코드	의미
200	요청 성공
400	잘못된 요청
401	인증 실패

코드	의미
404	요청한 자원이 없음
500	서버 내부 오류

실무에서는 API를 호출한 후 가장 먼저 상태 코드를 확인하는 것이 좋습니다.

2.6. JSON 데이터 가져오기

REST API는 대부분 데이터를 JSON 형식으로 반환합니다.

Response 객체의 `json()` 메서드를 호출하면 JSON 데이터를 Python 객체로 변환할 수 있습니다.

`practice/api_call.py`

```
import requests

url = "https://jsonplaceholder.typicode.com/users/1"

response = requests.get(url)

user = response.json()

print(user)
```

실행 결과

```
{
    "id": 1,
    "name": "Leanne Graham",
    "username": "Bret",
    "email": "Sincere@april.biz"
}
```

Python에서는 이 데이터를 **Dictionary**로 사용할 수 있습니다.

2.7. 필요한 데이터만 추출하기

JSON 전체를 사용하는 경우도 있지만, 필요한 값만 가져오는 경우가 더 많습니다.

예를 들어 이름과 이메일만 출력해 보겠습니다.

practice/api_call.py

```python
import requests

url = "https://jsonplaceholder.typicode.com/users/1"

response = requests.get(url)

user = response.json()

print(user["name"])
print(user["email"])
```

실행 결과

```
Leanne Graham
Sincere@april.biz
```

JSON 데이터는 일반 Python Dictionary처럼 사용할 수 있습니다.

2.8. API 호출 과정 정리

지금까지 작성한 코드는 내부적으로 다음과 같은 과정을 수행합니다.

```
① Python 프로그램 실행

↓

② requests.get()

↓

③ HTTP Request 생성

↓

④ API Server 전송

↓

⑤ HTTP Response 수신

↓

⑥ Response 객체 생성
```

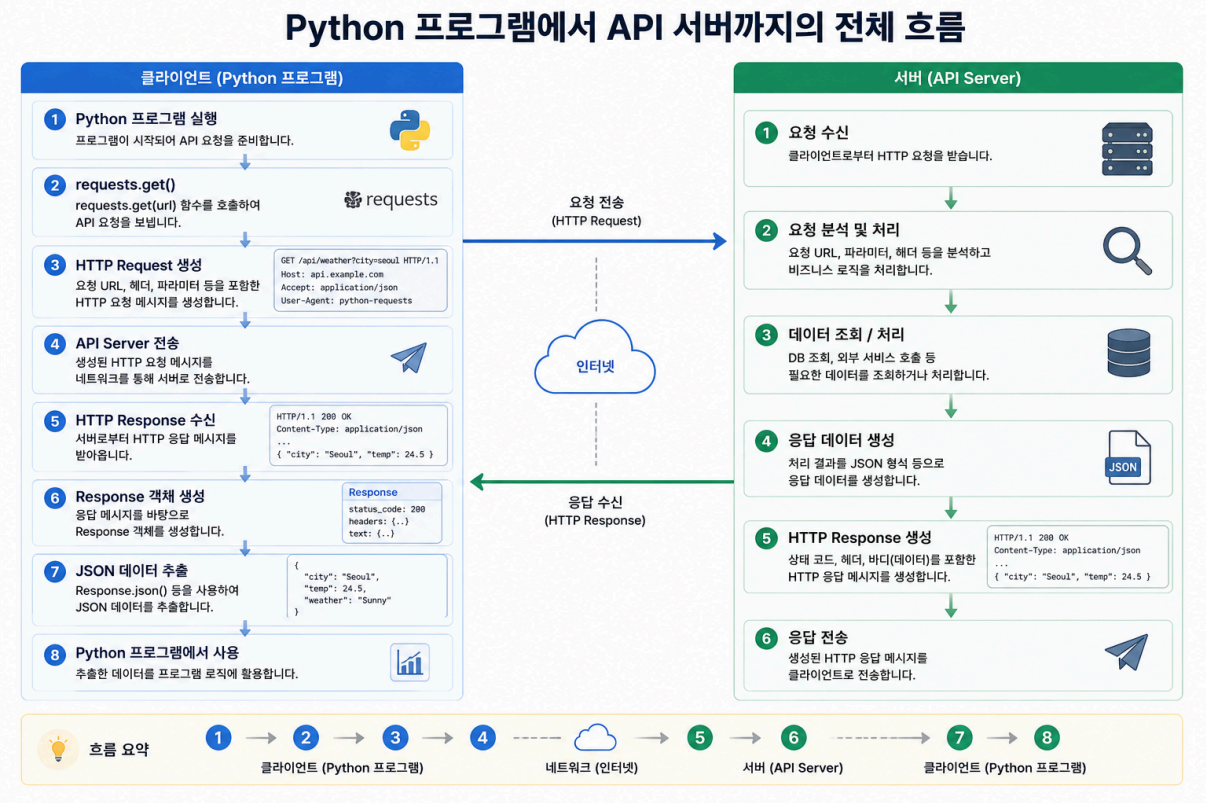
```
↓

⑦ JSON 데이터 추출

↓

⑧ Python 프로그램에서 사용
```

개발자는 `requests.get()` 한 줄만 작성하지만, 내부에서는 여러 단계의 네트워크 통신이 이루어집니다.



정리

이번 절에서는 Python의 `requests` 라이브러리를 이용하여 REST API를 호출하는 방법을 학습했습니다.

개념	설명
`requests`	Python에서 HTTP 요청을 보내는 라이브러리
`requests.get()`	GET 방식으로 API 호출
Response 객체	서버가 반환한 응답을 저장하는 객체

개념	설명
`status_code`	요청 성공 여부 확인
`json()`	JSON 데이터를 Python 객체로 변환

3. 날씨 API를 Agent의 Tool로 연결하기

3.1. Open-Meteo 날씨 API 소개

3.1.1. 사용할 API

이번 절에서는 §1~2에서 배운 REST API와 `requests`를 그대로 사용하여 **실시간 날씨를 조회하는 Tool**을 만듭니다.

사용할 API는 **Open-Meteo**입니다.

- 가입과 API Key가 필요 없습니다.
- 무료이며 호출 횟수 제한이 넉넉합니다.
- 위도·경도만 전달하면 현재 날씨와 예보를 JSON으로 반환합니다.

> **참고**
> 이 API는 2일차 n8n 과정에서도 같은 방식으로 사용합니다. 오늘은 Python 코드로, 내일은 노코드 노드로 같은 API를 호출하게 되므로 두 방식의 차이를 비교해 볼 수 있습니다.

3.1.2. URL 구조

Open-Meteo의 현재 날씨 조회 URL은 다음과 같습니다.

```
https://api.open-meteo.com/v1/forecast?latitude=37.5665&longitude=126.9780&current_wea
ther=true
```

길어 보이지만 1.8절에서 배운 쿼리 파라미터 세 개로 이루어져 있습니다.

파라미터	의미
`latitude=37.5665`	위도 (서울시청 기준)
`longitude=126.9780`	경도
`current_weather=true`	현재 날씨를 함께 달라는 요청

웹 브라우저 주소창에 이 URL을 그대로 입력해 보세요. JSON 응답이 화면에 표시됩니다. REST API는 브라우저로도 호출할 수 있다는 것을 확인하는 가장 간단한 방법입니다.

3.1.3. 응답 구조

응답 JSON에서 이번 실습에 사용할 부분은 `current_weather`입니다.

```
{
  "latitude": 37.5625,
  "longitude": 126.9875,
  "timezone": "GMT",
  "current_weather": {
    "time": "2026-07-03T05:00",
    "temperature": 24.3,
    "windspeed": 7.6,
    "winddirection": 180,
    "weathercode": 1
  }
}
```

필드	의미
`temperature`	현재 기온(℃)
`windspeed`	풍속(km/h)
`weathercode`	날씨 상태 코드 (0 맑음, 1~3 구름, 45·48 안개, 51~67 비, 71~77 눈, 80~82 소나기, 95 이상 뇌우)

3.2. Python으로 날씨 조회 함수 작성

3.2.1. 첫 번째 호출

2.3절과 같은 방식으로 호출합니다.

`practice/weather_test.py`

```
import requests

url = "https://api.open-meteo.com/v1/forecast"

params = {
```

```
    "latitude": 37.5665,
    "longitude": 126.9780,
    "current_weather": True,
}

response = requests.get(url, params=params)

print(response.status_code)
print(response.json())
```

실행

```
python practice\weather_test.py
```

`requests.get()`의 `params`에 딕셔너리를 전달하면 `?latitude=...&longitude=...` 형태의 쿼리 파라미터를 자동으로 만들어 줍니다. URL에 직접 이어 붙이는 것보다 읽기 쉽고 실수가 적습니다.

3.2.2. 필요한 값만 꺼내기

```
import requests

url = "https://api.open-meteo.com/v1/forecast"

params = {
    "latitude": 37.5665,
    "longitude": 126.9780,
    "current_weather": True,
}

response = requests.get(url, params=params)

data = response.json()

current = data["current_weather"]

print("기온 :", current["temperature"])
```

```
print("풍속 :", current["windspeed"])
print("날씨 코드 :", current["weathercode"])
```

실행 결과 (실행 시점에 따라 값은 달라집니다)

```
기온 : 24.3
풍속 : 7.6
날씨 코드 : 1
```

3.2.3. 도시 이름으로 조회하기

사용자는 "서울 날씨"라고 말하지, 위도·경도를 말하지 않습니다.

도시 이름을 좌표로 바꾸는 딕셔너리를 만듭니다.

```
CITY_COORDS = {
    "서울": (37.5665, 126.9780),
    "부산": (35.1796, 129.0756),
    "대구": (35.8714, 128.6014),
    "광주": (35.1595, 126.8526),
    "대전": (36.3504, 127.3845),
    "제주": (33.4996, 126.5312),
}
```

> **참고**
> 다른 도시를 추가하려면 Google 지도에서 원하는 위치를 우클릭하면 맨 위에 위도·경도가 표시됩니다.

3.2.4. 함수로 만들기

Tool로 등록하려면 함수 형태여야 합니다. 날씨 코드를 사람이 읽을 수 있는 문장으로 바꾸는 작은 함수도 함께 만듭니다.

```
import requests

CITY_COORDS = {
    "서울": (37.5665, 126.9780),
    "부산": (35.1796, 129.0756),
    "대구": (35.8714, 128.6014),
```

```
    "광주": (35.1595, 126.8526),
    "대전": (36.3504, 127.3845),
    "제주": (33.4996, 126.5312),
}


def describe_weather_code(code: int) -> str:
    if code == 0:
        return "맑음"
    if code <= 3:
        return "구름 조금"
    if code in (45, 48):
        return "안개"
    if 51 <= code <= 67:
        return "비"
    if 71 <= code <= 77:
        return "눈"
    if 80 <= code <= 82:
        return "소나기"
    if code >= 95:
        return "뇌우"
    return "알 수 없음"


def get_current_weather(city: str) -> dict:
    if city not in CITY_COORDS:
        return {"error": f"지원하지 않는 도시입니다: {city}"}

    lat, lon = CITY_COORDS[city]

    response = requests.get(
        "https://api.open-meteo.com/v1/forecast",
        params={"latitude": lat, "longitude": lon, "current
_weather": True},
        timeout=10,
    )

    current = response.json()["current_weather"]
```

```
    return {
        "city": city,
        "temperature": current["temperature"],
        "windspeed": current["windspeed"],
        "condition": describe_weather_code(current["weather
code"]),
    }


print(get_current_weather("서울"))
print(get_current_weather("제주"))
```

실행 결과 예시

```
{'city': '서울', 'temperature': 24.3, 'windspeed': 7.6, 'con
dition': '구름 조금'}
{'city': '제주', 'temperature': 26.1, 'windspeed': 12.4, 'co
ndition': '맑음'}
```

반환값이 Dictionary이므로 Agent는 어느 값이 기온이고 어느 값이 풍속인지 구분할 수 있습니다.

3.2.5. 예외 처리

네트워크가 끊기거나 서버가 응답하지 않으면 `requests.get()`에서 오류가 발생합니다.

이런 실행 중 오류를 예외(Exception)라고 합니다. Tool 안에서 예외가 발생하면 Agent 전체가 멈추므로, `try` / `except` 문으로 예외를 잡아서(예외 처리, Exception Handling) 메시지로 반환합니다.

```
def get_current_weather(city: str) -> dict:
    if city not in CITY_COORDS:
        return {"error": f"지원하지 않는 도시입니다: {city}"}

    lat, lon = CITY_COORDS[city]

    try:
        response = requests.get(
```

```
            "https://api.open-meteo.com/v1/forecast",
            params={"latitude": lat, "longitude": lon, "cur
rent_weather": True},
            timeout=10,
        )
        response.raise_for_status()

        current = response.json()["current_weather"]

        return {
            "city": city,
            "temperature": current["temperature"],
            "windspeed": current["windspeed"],
            "condition": describe_weather_code(current["wea
thercode"]),
        }

    except requests.RequestException:
        return {"error": "날씨 정보를 가져올 수 없습니다. 잠시 후
다시 시도하세요."}
```

`raise_for_status()`는 상태 코드가 400 이상이면 예외를 발생시킵니다. 2.5절에서 배운 상태 코드 확인을 한 줄로 처리하는 방법입니다.

> **참고**
> Tool이 오류 상황에서 `{"error": "..."}`처럼 설명이 담긴 값을 반환하면, Agent는 그 내용을 읽고 사용자에게 상황을 안내할 수 있습니다. 예외를 그대로 터뜨리는 것보다 이 방식이 사용자 경험에 유리합니다.

3.3. Weather Tool 만들기 및 Agent 등록

3.3.1. Tool 파일 작성

3.2절에서 만든 함수를 `tools` 폴더로 옮기고, Docstring을 추가합니다.

`tools/weather_tool.py`

```
import requests

CITY_COORDS = {
```

```
    "서울": (37.5665, 126.9780),
    "부산": (35.1796, 129.0756),
    "대구": (35.8714, 128.6014),
    "광주": (35.1595, 126.8526),
    "대전": (36.3504, 127.3845),
    "제주": (33.4996, 126.5312),
}


def describe_weather_code(code: int) -> str:
    if code == 0:
        return "맑음"
    if code <= 3:
        return "구름 조금"
    if code in (45, 48):
        return "안개"
    if 51 <= code <= 67:
        return "비"
    if 71 <= code <= 77:
        return "눈"
    if 80 <= code <= 82:
        return "소나기"
    if code >= 95:
        return "뇌우"
    return "알 수 없음"


def get_current_weather(city: str) -> dict:
    """
    지정한 도시의 현재 날씨(기온, 풍속, 날씨 상태)를 조회합니다.

    Args:
        city: 도시 이름. 지원 도시: 서울, 부산, 대구, 광주, 대전, 제
주

    Returns:
        city, temperature(°C), windspeed(km/h), condition을
담은 dict.
```

```
        지원하지 않는 도시이거나 조회에 실패하면 error 키에 설명을 담아
반환합니다.
    """

    if city not in CITY_COORDS:
        return {"error": f"지원하지 않는 도시입니다: {city}"}

    lat, lon = CITY_COORDS[city]

    try:
        response = requests.get(
            "https://api.open-meteo.com/v1/forecast",
            params={"latitude": lat, "longitude": lon, "cur
rent_weather": True},
            timeout=10,
        )
        response.raise_for_status()

        current = response.json()["current_weather"]

        return {
            "city": city,
            "temperature": current["temperature"],
            "windspeed": current["windspeed"],
            "condition": describe_weather_code(current["wea
thercode"]),
        }

    except requests.RequestException:
        return {"error": "날씨 정보를 가져올 수 없습니다. 잠시 후
다시 시도하세요."}
```

3.3.2. Docstring 작성 포인트

Chapter 3에서 배운 대로 LLM은 Docstring을 읽고 Tool을 선택하고, 매개변수를 채웁니다.

이 Tool의 Docstring에는 두 가지가 명시되어 있습니다.

- **지원 도시 목록**: "인천 날씨 알려줘"처럼 지원하지 않는 도시를 물었을 때 LLM이 미리 알 수 있습니다.
- **반환값 구조**: `error` 키가 있을 수 있다는 것을 알면 LLM이 오류 상황을 자연스럽게 안내합니다.

`describe_weather_code`는 Tool로 등록하지 않는 보조 함수이므로 Docstring이 없어도 됩니다.

3.3.3. Agent에 등록

`agent/agent.py`

```
from google.adk.agents import Agent

from tools.time_tool import get_current_time
from tools.calculator_tool import calculate
from tools.dice_tool import roll_dice
from tools.exchange_rate_tool import get_exchange_rate
from tools.memo_tool import save_memo, read_memo
from tools.weather_tool import get_current_weather

root_agent = Agent(
    model="gemini-3.1-flash-lite",
    name="multi_tool_agent",
    description="여러 Tool을 사용하는 Agent",
    instruction="""
    사용자의 질문에 적절한 Tool을 선택하여 답변하세요.
    """,
    tools=[
        get_current_time,
        calculate,
        roll_dice,
        get_exchange_rate,
        save_memo,
        read_memo,
        get_current_weather
    ]
)
```

> **참고**
> Chapter 3의 심화 실습을 건너뛰었다면 `get_exchange_rate`, `save_memo`, `read_memo` 줄은 제외하고 작성합니다.

3.3.4. 실습 4-1. 날씨 조회 Tool 호출

프로젝트 루트에서 `adk web`을 실행하고 `agent`를 선택한 뒤 다음을 질문합니다.

```
서울 날씨 알려줘.
```

Events 탭에서 다음을 확인합니다.

- `functionCall`에 `get_current_weather`가 있고, `city` 인자에 `"서울"`이 들어갔는지
- `functionResponse`에 기온·풍속·날씨 상태가 담겨 있는지

이어서 다음 질문도 입력합니다.

```
제주는 지금 바람 많이 불어?
```

```
인천 날씨는?
```

두 번째 질문에서 LLM이 Docstring의 지원 도시 목록을 근거로 "인천은 지원하지 않는다"고 답하는지, 또는 Tool을 호출해 `error`를 받은 뒤 안내하는지 Events 탭으로 확인합니다. 어느 쪽이든 사용자에게는 올바른 안내가 나가야 합니다.

3.3.5. 여러 도시 한 번에 조회

```
서울이랑 부산 중에 어디가 더 따뜻해?
```

LLM은 필요한 경우 같은 Tool을 **여러 번** 호출합니다. Events 탭에 `get_current_weather`가 두 번 나타나고, 두 결과를 비교해 답변하는 것을 확인합니다.

3.4. 날씨 예보 Tool 추가

3.4.1. 예보 API 파라미터

Open-Meteo는 같은 URL에 파라미터를 바꾸면 일별 예보도 반환합니다.

파라미터	의미
`daily=temperature_2m_max,temperature_2m_min,precipitation_sum`	일별 최고기온, 최저기온, 강수량
`forecast_days=3`	오늘부터 3일치 (최대 16)
`timezone=Asia/Seoul`	한국 시간 기준 날짜

응답의 `daily` 항목은 날짜 배열과 값 배열이 나란히 들어 있습니다.

```
{
  "daily": {
    "time": ["2026-07-03", "2026-07-04", "2026-07-05"],
    "temperature_2m_max": [29.1, 30.4, 27.8],
    "temperature_2m_min": [21.3, 22.0, 20.5],
    "precipitation_sum": [0.0, 2.4, 11.2]
  }
}
```

3.4.2. 예보 Tool 작성

`tools/weather_tool.py`에 함수를 추가합니다.

```
def get_weather_forecast(city: str, days: int = 3) -> dict:
    """
    지정한 도시의 일별 날씨 예보(최고기온, 최저기온, 강수량)를 조회합니다.

    Args:
        city: 도시 이름. 지원 도시: 서울, 부산, 대구, 광주, 대전, 제주
        days: 조회할 일수. 오늘을 포함하며 1~7 사이의 값. 기본값 3

    Returns:
        날짜(YYYY-MM-DD)를 키로, max_temp(°C), min_temp(°C), rain_mm를 값으로 갖는 dict.
        지원하지 않는 도시이거나 조회에 실패하면 error 키에 설명을 담아 반환합니다.
    """
```

```
    if city not in CITY_COORDS:
        return {"error": f"지원하지 않는 도시입니다: {city}"}

    days = max(1, min(days, 7))

    lat, lon = CITY_COORDS[city]

    try:
        response = requests.get(
            "https://api.open-meteo.com/v1/forecast",
            params={
                "latitude": lat,
                "longitude": lon,
                "daily": "temperature_2m_max,temperature_2m
_min,precipitation_sum",
                "forecast_days": days,
                "timezone": "Asia/Seoul",
            },
            timeout=10,
        )
        response.raise_for_status()

        daily = response.json()["daily"]

        result = {}
        for i, date in enumerate(daily["time"]):
            result[date] = {
                "max_temp": float(daily["temperature_2m_ma
x"][i]),
                "min_temp": float(daily["temperature_2m_mi
n"][i]),
                "rain_mm": float(daily["precipitation_sum"]
[i]),
            }

        return result
```

```
    except requests.RequestException:
        return {"error": "예보 정보를 가져올 수 없습니다. 잠시 후
다시 시도하세요."}
```

> **주의**
> Tool의 반환값은 반드시 JSON으로 변환 가능한 형태(문자열 키, `str` · `int` · `float` · `bool` · `list` · `dict`)여야 합니다. 날짜 객체나 numpy 숫자처럼 변환되지 않는 값이 섞이면 Tool 결과를 LLM에 전달하는 과정에서 오류가 발생합니다. 위 코드에서 `float()` 로 감싼 것은 이 때문입니다.

3.4.3. Instruction 개선과 등록

Tool이 두 개가 되었으므로 Instruction에서 사용 규칙을 명확히 합니다.

`agent/agent.py`

```
from google.adk.agents import Agent

from tools.time_tool import get_current_time
from tools.calculator_tool import calculate
from tools.dice_tool import roll_dice
from tools.exchange_rate_tool import get_exchange_rate
from tools.memo_tool import save_memo, read_memo
from tools.weather_tool import get_current_weather, get_wea
ther_forecast

root_agent = Agent(
    model="gemini-3.1-flash-lite",
    name="multi_tool_agent",
    description="여러 Tool을 사용하는 Agent",
    instruction="""
    당신은 AI 비서입니다.

    사용자의 질문을 이해한 후 적절한 Tool을 선택하여 실행하세요.

    Tool을 사용할 수 있는 경우에는 직접 추측하지 말고 반드시 Tool을 사
용하세요.

    "지금", "현재" 날씨는 get_current_weather를,
```

```
    "내일", "주말", "며칠간" 같은 예보는 get_weather_forecast를
사용하세요.

    Tool 실행 결과를 이해하기 쉬운 자연스러운 한국어로 설명하세요.

    필요하면 여러 Tool을 순차적으로 사용할 수 있습니다.
    """,
    tools=[
        get_current_time,
        calculate,
        roll_dice,
        get_exchange_rate,
        save_memo,
        read_memo,
        get_current_weather,
        get_weather_forecast,
    ],
)
```

3.4.4. 실습 4-2. 여러 Tool 자동 선택 확인

Agent를 실행한 후 다음 질문을 차례로 입력하고, Events 탭에서 어떤 Tool이 어떤 인자로 호출되는지 확인합니다.

질문	기대 동작
부산 지금 날씨 어때?	`get_current_weather(city="부산")`
서울 내일 비 와?	`get_weather_forecast(city="서울", days=2)` 또는 3, 응답에서 내일 날짜의 rain_mm로 판단
제주 이번 주 최고기온 알려줘	`get_weather_forecast(city="제주", days=7)`
대구 날씨 확인하고 지금 몇 시인지도 알려줘	`get_current_weather`와 `get_current_time` 두 Tool 호출
오늘 서울 최고기온이랑 최저기온 차이 계산해줘	`get_weather_forecast` 호출 후 `calculate` 호출

마지막 질문에서는 첫 번째 Tool의 결과를 두 번째 Tool의 입력으로 넘기는 과정이 Events 탭에 순서대로 기록됩니다. 이것이 Chapter 1에서 설명한 "여러 작업을 연결하는 Agent"의 실제 동작입니다.

3.5. API Key가 필요한 API의 경우

3.5.1. 대부분의 API는 인증이 필요하다

Open-Meteo는 API Key 없이 사용할 수 있지만, 실무에서 사용하는 API는 대부분 **API Key**를 요구합니다.

- 공공데이터포털(기상청, 교통 정보 등)
- Google Maps, 네이버 지도
- 환율·주가 서비스
- Slack, Notion, GitHub

API Key는 "누가 호출했는지"를 서버가 확인하는 수단이며, 호출 횟수 제한과 과금의 기준이 됩니다.

3.5.2. API Key를 코드에 넣지 않는다

Chapter 2에서 Gemini API Key를 `.env`에 저장한 것과 같은 방식을 사용합니다.

`.env`

```
GOOGLE_API_KEY=AIzaSy****************************
WEATHER_API_KEY=여기에_발급받은_키
```

Tool 코드

```python
import os
import requests
from dotenv import load_dotenv

load_dotenv()

API_KEY = os.getenv("WEATHER_API_KEY")


def get_weather_with_key(city: str) -> dict:
    """
    (예시) API Key가 필요한 날씨 서비스를 호출합니다.
    """
```

```
    response = requests.get(
        "https://api.example-weather.com/v1/current",
        params={"city": city, "appid": API_KEY},
        timeout=10,
    )
    response.raise_for_status()

    return response.json()
```

API Key를 전달하는 위치는 서비스마다 다릅니다.

전달 방식	예시
쿼리 파라미터	`params={"appid": API_KEY}`
HTTP 헤더	`headers={"Authorization": f"Bearer {API_KEY}"}`

헤더(Header)는 요청 본문과 별도로 보내는 부가 정보이고, `Bearer`는 "이 토큰을 가진 (bearer) 사람의 요청"이라는 뜻의 인증 방식 이름입니다. 어느 방식이든 공통 원칙은 같습니다. **키는 `.env`에만 두고, 코드에는 `os.getenv()`로 읽어 오는 줄만 남깁니다.** Chapter 7의 실습 7-4에서 헤더 방식을 실제로 사용합니다.

> **주의**
> `.env` 파일은 `.gitignore`에 추가하여 GitHub 등 외부에 올라가지 않도록 합니다. 실습 중 화면을 공유할 때도 `.env` 파일은 열지 않습니다.

정리

이번 장에서는 REST API를 이해하고, Python으로 호출한 뒤, 그 함수를 Agent의 Tool로 등록하는 전체 과정을 학습했습니다.

개념	설명
REST API	HTTP Request/Response로 데이터를 주고받는 방식
쿼리 파라미터	`?key=value&key2=value2` 형태로 요청 조건 전달. `requests`의 `params`로 작성
`response.json()`	JSON 응답을 Python dict로 변환
Weather Tool	도시명 → 좌표 변환 → Open-Meteo 호출 → dict 반환
예외 처리	Tool 안에서 오류를 잡아 `error` 키로 반환
JSON 변환 가능 반환값	문자열 키와 기본 자료형만 사용

개념	설명
API Key 관리	`.env`에 저장하고 `os.getenv()`로 읽기

체크포인트 질문

1. AI Agent가 날씨·환율·주가 같은 정보를 사용하려면 왜 API가 필요한가요? → **이 데이터는 외부 시스템이 관리하므로, 프로그램 간에 데이터를 주고받는 약속(API)을 통해 요청해야 하기 때문**
2. `requests.get(url)`의 반환값에서 JSON 데이터를 Python 딕셔너리로 꺼내는 메서드는 무엇인가요? → **`response.json()`**
3. `https://api.open-meteo.com/v1/forecast?latitude=37.5&longitude=127&current_weather=true`에서 `?` 뒤의 부분을 무엇이라고 하며, `requests`에서는 어떻게 작성하나요? → **쿼리 파라미터. `requests.get(url, params={...})`의 딕셔너리로 작성**
4. 사용자가 "서울 날씨"라고 물었을 때 Tool이 위도·경도를 알 수 있는 이유는 무엇인가요? → **Tool 안의 `CITY_COORDS` 딕셔너리가 도시명을 좌표로 변환하기 때문. LLM은 도시명만 인자로 전달함**
5. Tool 함수가 날짜 객체나 numpy 숫자처럼 JSON으로 변환되지 않는 값을 반환하면 어떤 문제가 생기나요? → **Tool 결과를 LLM에 전달하는 과정에서 오류가 발생함. 문자열 키와 기본 자료형(`str`, `float`, `int`)으로 변환해서 반환해야 함**
6. "부산 지금 날씨"와 "부산 이번 주 예보"는 각각 어떤 Tool이 선택되며, 그 근거는 무엇인가요? → **전자는 `get_current_weather`, 후자는 `get_weather_forecast`. LLM이 Docstring과 Instruction의 사용 규칙을 읽고 질문 의도에 맞는 Tool을 선택함**
7. API Key가 필요한 API를 Tool로 만들 때 키는 어디에 두어야 하나요? → **`.env` 파일. 코드에서는 `os.getenv()`로 읽어 오며, 키 값을 코드에 직접 쓰지 않음**